

BOM ↔ Engineering Drawing Automation

Approach Study and Methodology Report

Focus: Drawing → BOM and BOM → Drawing

1. Project Objective

The project aims to automate bidirectional conversion between Engineering Drawings and Bill of Materials (BOM). Drawing → BOM extracts component information from an engineering drawing and converts it into a structured BOM. BOM → Drawing reads structured BOM information, identifies component types, applies dimensions and parameters, and generates a corresponding engineering drawing.

2. Four Approaches Considered

Approach	Technologies	Main Idea	Status / Purpose
1. OpenCV + Tesseract OCR	OpenCV, Tesseract	Pre-process the drawing and extract text, dimensions and section labels using OCR.	Tested on a real drawing; mainly read title text and failed to reliably extract dimensions and section labels.
2. Gemini Vision + Python + DXF	Gemini Vision API, Python, engineering formulas, DXF/CAD	Use a vision model to understand the drawing and return JSON; process JSON in Python, calculate engineering values and generate DXF from the BOM.	Current approach.
3. Python + Streamlit	Python, Streamlit, OCR/OpenCV and Python processing	Provide a simple interface for uploading drawings/BOMs, processing them with Python and producing structured outputs.	Alternative prototype approach.
4. AI + Engineering Template Library	AI/Vision, CAD templates, Python, DXF/CAD	Identify standard engineering components using reusable templates, extract parameters and use templates for BOM extraction and drawing generation.	Alternative; library research performed, but ready-made templates were not found for all required project-specific parts.

3. Approach 1 – OpenCV + Tesseract OCR

3.1 Drawing → BOM

The first approach used OpenCV to preprocess the engineering drawing and Tesseract OCR to read information. The intended process was to enhance the drawing, detect text, dimensions and section labels, and convert the extracted information into a BOM.

Flow: Engineering Drawing → OpenCV preprocessing → Tesseract OCR → Text/Dimension extraction → Component information → Structured BOM

This approach was tested on a real engineering drawing. Tesseract primarily detected the title text and did not reliably identify the dimensions and section labels required for BOM generation. Therefore, it was not considered reliable enough as the main solution.

3.2 BOM → Drawing

For the reverse direction, a rule/template-based Python process can read the structured BOM, map component information to drawing logic, and generate a DXF/SVG/PDF drawing.

Flow: BOM → Read/parse BOM → Component identification → Parameter insertion → Drawing generation → DXF/SVG/PDF

3.3 Advantages and Limitations

Advantages: simple, open-source, lightweight and easy to prototype.

Limitation: OCR is not reliable for complex engineering drawings containing dimensions, section labels, symbols and overlapping graphical elements. The real-drawing test demonstrated this limitation.

4. Approach 2 – Gemini Vision + Python + DXF Generation

4.1 Drawing → BOM

The current approach uses Google's Gemini Vision API as a vision model to interpret the engineering drawing. The drawing is provided to the model and component information is extracted in structured JSON format. Python then processes the JSON and applies standard steel engineering formulas to calculate actual component weights and other derived values required for the BOM.

Current flow: Engineering Drawing → Gemini Vision → Structured JSON → Python processing → Engineering calculations → Weight/derived parameters → Structured BOM

4.2 BOM → Drawing

After the BOM is produced and processed, the BOM information is used by Python to generate a DXF file. The generated DXF is then opened/pasted into CAD software to obtain the corresponding engineering drawing.

Current reverse workflow: Structured BOM → Python BOM processing → DXF generation → DXF file → CAD software → Engineering Drawing

This gives a practical chain: Drawing → BOM → DXF → CAD Drawing.

4.3 Advantages and Limitations

Advantages: strong visual understanding, structured JSON output, easier component-level extraction, and integration with Python engineering calculations and DXF generation.

Limitations: model output must be validated because extracted dimensions or component interpretations may be incorrect or incomplete. The approach also depends on the vision API.

5. Approach 3 – Python + Streamlit Automation

5.1 Drawing → BOM

This approach focuses on building the prototype around Python, with Streamlit providing the user interface. The user uploads an engineering drawing through Streamlit. Python performs image processing, OCR/drawing analysis, extraction, engineering calculations and BOM generation.

Flow: Engineering Drawing → Streamlit upload → Python processing → OCR/OpenCV or analysis → Information extraction → Engineering calculations → BOM → Download

5.2 BOM → Drawing

The same Streamlit interface can accept an Excel/CSV BOM. Python reads and validates the BOM, identifies the component type, applies drawing-generation rules and creates a DXF/SVG/PDF output that can be opened in CAD.

Flow: BOM → Streamlit upload → Pandas/Python parsing → Component identification → Drawing generation → DXF → CAD

5.3 Advantages and Limitations

Advantages: simple interface, single Python ecosystem, fast prototype development and easy demonstration.

Limitation: Streamlit itself does not solve engineering drawing interpretation. OCR, computer vision, CAD parsing and drawing-generation logic are still required.

6. Approach 4 – AI + Engineering Template Library

6.1 Template Library Research

This approach uses AI for interpretation where required and a reusable engineering template library for standard component types. Planned templates include Plate, UB/I-Section, Angle, Channel, Pipe, Flat Bar, Gusset Plate and similar structural components.

The team researched available CAD/template libraries to determine whether ready-made components could be directly reused. Standard structural profiles are available from external CAD libraries, but a complete ready-made library containing all project-specific components and drawing formats required for the IJGEC drawing was not found.

Therefore, external libraries can be used as references or sources for standard geometry, while missing project-specific components would need to be created or customized within the project.

6.2 Drawing → BOM

AI/Vision first identifies the component or section in the drawing. The system matches it to the closest component template. Dimensions, material, quantity and other parameters are extracted and mapped to the BOM data model. Engineering rules are then used for normalization, calculations and validation.

Flow: Drawing → AI/Vision → Component identification → Template matching → Parameter extraction → Engineering rules → Validation → Structured BOM

6.3 BOM → Drawing

For BOM → Drawing, the component type from the BOM selects a template. BOM parameters are inserted into the template and the resulting geometry is exported as DXF/SVG/PDF.

Flow: BOM → Component identification → Template selection → Insert dimensions/material/parameters → Generate DXF/SVG/PDF → CAD

6.4 Advantages and Limitations

Advantages: reusable components, consistent outputs, reduced need to recreate common geometry and better control over engineering drawing generation.

Limitations: building and maintaining templates requires initial effort. The research showed that some project-specific components were not available as ready-made templates, so those components would have to be created or customized.

7. Comparison of the Four Approaches

Criteria	Approach 1	Approach 2	Approach 3	Approach 4
Drawing interpretation	OCR-based	Gemini Vision	Depends on processing method	AI + templates
Current status	Tested; poor key-field extraction	Current approach	Alternative prototype	Alternative; library research completed
Drawing → BOM	OCR extraction	AI JSON + Python	Python-based	AI + template matching
BOM → Drawing	Rules/templates	Python → DXF → CAD	Python → DXF/CAD	Parametric templates → DXF/CAD
Main limitation	OCR accuracy	Requires validation	Underlying analysis still required	Not all required parts have ready-made templates

8. Overall Bidirectional Workflow

Drawing → BOM: Drawing input → preprocessing/vision → component and parameter extraction → normalization → engineering calculations → validation → structured BOM.

BOM → Drawing: BOM input → parsing/normalization → component identification → template/rule selection → parameter insertion → DXF/SVG/PDF generation → CAD.

9. Current Direction

Based on actual testing, OpenCV + Tesseract was found insufficient for reliable extraction from the real engineering drawing. The current implementation therefore uses Gemini Vision for drawing interpretation, Python for structured processing and engineering calculations, and DXF generation for the BOM → Drawing direction.

The template-library approach remains useful for standard structural sections and repetitive components. However, because ready-made libraries did not cover every required project-specific component, a hybrid strategy would require creating or customizing the missing templates.